Memora

Input-to-Graph Pipeline

Deep Technical Analysis: Graph Construction,
AI Agent Architecture & Palantir-Class Intelligence

Technical Deep Dive Document

How Raw Input Becomes a Living Knowledge Graph

Date:	February 25, 2026
Status:	Technical Reference
Scope:	Graph Pipeline + Agent Architecture + Palantir Analysis
Audience:	Engineering Team & Architecture Review

CONFIDENTIAL — Internal Distribution Only

Contents

1 Thesis: Memora as Your Personal Palantir **3**
1.1 The Palantir Architecture (Enterprise/Government Scale) . . . 3
1.2 Mapping Palantir to Memora . . . 4

2 The Complete Input-to-Graph Pipeline **5**
2.1 Pipeline Overview . . . 5

3 Stage-by-Stage Breakdown **6**
3.1 Stage 1: Raw Input Capture . . . 6
3.2 Stage 2: Preprocessing . . . 6
3.3 Stage 3: The Archivist — LLM Extraction Engine . . . 7
3.3.1 What the Archivist Does . . . 7
3.3.2 Constrained Output via Pydantic Schemas . . . 7
3.3.3 The Archivist's System Prompt Architecture . . . 8
3.4 Stage 4: Entity Resolution . . . 9
3.4.1 Resolution Strategy . . . 9
3.4.2 Entity Resolution Outcomes . . . 9
3.5 Stage 5: Graph Proposal Assembly . . . 10
3.6 Stage 6: Validation Gate . . . 10
3.7 Stage 7: Human Review / Auto-Approve . . . 11
3.8 Stage 8: Graph Commit . . . 11
3.9 Stage 9: Post-Commit Processing . . . 11

4 Graph Construction: The Ontology **12**
4.1 Node Types (The Entities) . . . 12
4.2 Shared Node Properties (The Metadata Layer) . . . 12
4.3 Edge Types (The Relationships) . . . 13
4.4 Context Networks (The Subgraphs) . . . 14

5 How AI Agents Operate on the Graph **16**
5.1 The Agent Architecture: Palantir AIP vs. Memora Council . . . 16
5.2 Agent 1: The Archivist (Graph Writer) . . . 16
5.3 Agent 2: The Strategist (Graph Reader + Analyst) . . . 16
5.4 Agent 3: The Researcher (Internet Bridge) . . . 17
5.5 The Orchestrator: Agent Coordination . . . 17
5.6 Council Decision Pattern (High-Stakes Queries) . . . 17

6 The Truth Layer: Fact Verification Pipeline **19**
6.1 Why It Exists . . . 19
6.2 The Fact-Check Gate . . . 19

7 Background Mechanics: The Living Graph Engine **20**
7.1 Decay Scoring (Daily) . . . 20
7.2 Bridge Discovery (Daily + Per-Capture) . . . 20
7.3 Network Health Scoring (Every 6 Hours) . . . 20
7.4 Spaced Repetition (Daily) . . . 20
7.5 Gap Detection (Weekly) . . . 21

8 Palantir-Class Surveillance for Personal Intelligence **22**
8.1 The Surveillance Analogy 22
8.2 What Memora Surveils 22
8.3 The Daily Briefing as Intelligence Report 22
8.4 The Web of Connections 23

9 The Adaptive RAG Pipeline **24**

10 Technical Architecture Comparison: Palantir vs. Memora **26**

11 Conclusion: Building Your Personal Intelligence Agency **27**

1 Thesis: Memora as Your Personal Palantir

Key Insight

Palantir Technologies built a $250B+ company on a single idea: **take siloed, disconnected data from dozens of sources and weave it into a unified knowledge graph where entities (people, places, events, organizations) are linked by typed relationships**—then let AI agents reason over that graph to surface hidden connections, predict outcomes, and recommend actions.

Memora is that same architecture applied to a single human life. Instead of government databases, the inputs are voice memos, text captures, and screenshots. Instead of intelligence analysts, the consumers are you—making decisions about your career, relationships, health, finances, and ventures. The graph construction pipeline, the ontology, the entity resolution, the agent-driven reasoning—it is all structurally identical to what Palantir does at nation-state scale.

1.1 The Palantir Architecture (Enterprise/Government Scale)

Palantir's platform rests on three foundational pillars:

1. **The Ontology** — A semantic layer that maps raw data (tables, files, streams) to real-world entities. In Palantir's words: "The Ontology sits on top of the digital assets integrated into the Palantir platform and connects them to their real-world counterparts." It serves as a **digital twin** of the organization—containing *Object Types* (entities like "Employee," "Aircraft," "Threat Actor"), *Properties* (attributes of each entity), and *Link Types* (relationships between entities).

2. **The Data Pipeline** — Raw data from hundreds of sources (databases, APIs, sensor feeds, documents, emails) flows through Palantir Foundry's Pipeline Builder, where it is cleaned, deduplicated, enriched, and transformed into ontology objects. Entity resolution links records across sources: the same person appearing in an arrest log, a license plate database, and a phone record is resolved into a single node.

3. **AI Agents (AIP)** — Palantir's Artificial Intelligence Platform deploys LLM-powered agents that operate *on top of* the ontology. Agents have three context sources: ontology context (structured entity data), document context (unstructured text with semantic search), and function-backed context (custom retrieval). Agents use **tools** to read from and write to the ontology—executing actions, running queries, and automating workflows.

Palantir Parallel

Palantir's ontology has three layers: (1) **Semantic Layer** — object types, properties, link types that define what exists and how things relate; (2) **Kinetic Layer** — action types and functions that enable *change* while complying with governance controls; (3) **Dynamic Layer** — security rules, permissions, and access controls that govern who can see and modify what. This three-layer model ensures the graph is not just a static map but an **operational system** where AI and humans can take action.

1.2 Mapping Palantir to Memora

Concept	Palantir (Enterprise)	Memora (Personal)
Data Sources	100s of databases, APIs, sensors, documents	Text captures, voice memos, screenshots
Ontology / Schema	Object Types (Employee, Aircraft, Threat Actor)	Node Types (PERSON, EVENT, COMMITMENT, DECISION, GOAL, IDEA)
Properties	Name, ID, coordinates, classifications	Confidence, timestamps, embeddings, network memberships
Link Types	"works_at," "traveled_to," "communicated_with"	STRUCTURAL, ASSOCIATIVE, PROVENANCE, TEMPORAL, PERSONAL, SOCIAL, NETWORK
Entity Resolution	Cross-database record linkage (same person in 5 DBs)	The Archivist deduplicates entities across captures
Data Pipeline	Foundry Pipeline Builder (Spark/Flink transforms)	Archivist extraction + Pydantic schema validation
AI Agents	AIP Agent Studio (LLM + ontology + tools)	AI Council (Archivist + Strategist + Researcher)
Agent Tools	Ontology actions, object queries, custom functions	MCP servers (search, scraping, arXiv, graph/vector DB)
Digital Twin	Organization's operational reality	Your life's complete context graph
Intelligence Output	Threat detection, fraud alerts, operational decisions	Daily briefings, bridge discovery, decision recommendations
Surveillance	Track entities, movements, communications, networks	Track commitments, relationships, goals, health, momentum

Key Insight

The structural parallel is exact. Palantir ingests raw data → resolves entities → builds an ontology graph → deploys AI agents that reason over the graph → surfaces hidden connections → recommends actions. Memora follows the identical pipeline but the "nation" being surveilled is **your own life**, and the "intelligence analyst" is **you**.

2 The Complete Input-to-Graph Pipeline

This section traces a single piece of information from the moment it enters Memora to the moment it becomes committed nodes and edges in the knowledge graph. Every step is detailed.

2.1 Pipeline Overview

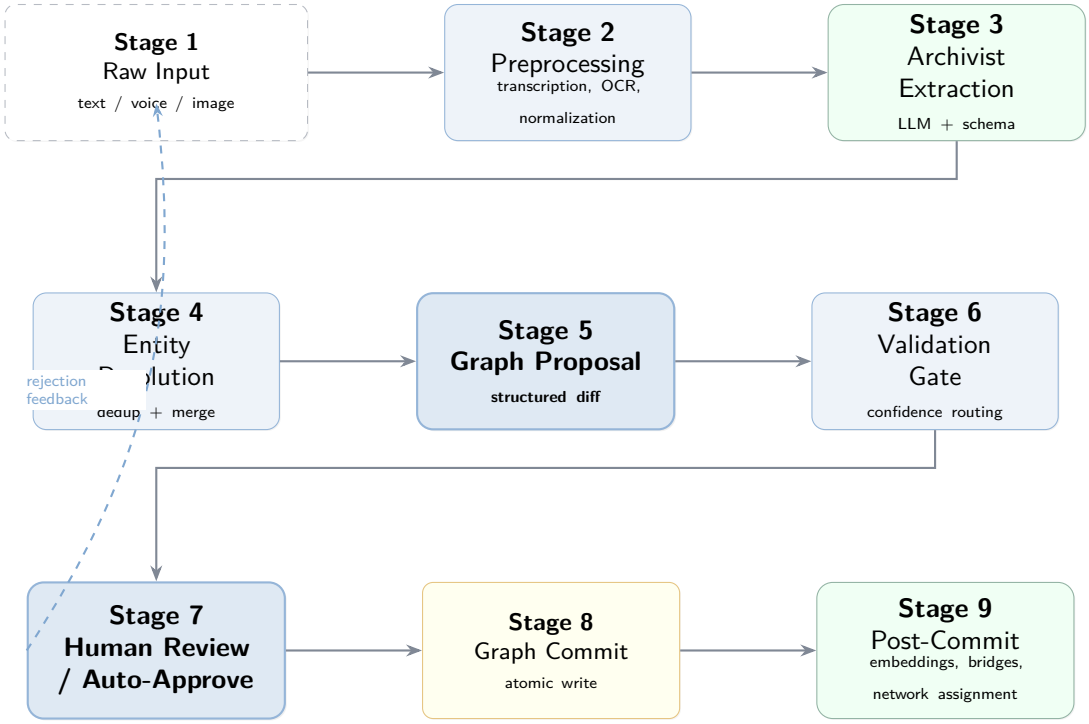

Figure 1: The complete 9-stage input-to-graph pipeline. Every piece of information traverses all nine stages before becoming committed knowledge. The pipeline is designed to mirror Palantir's Foundry data integration pipeline: raw ingestion → transformation → entity resolution → ontology mapping → governance gate → commit.

3 Stage-by-Stage Breakdown

3.1 Stage 1: Raw Input Capture

Two input modalities only: text and images. This is a deliberate constraint—the user is the sole source of truth. No silent calendar syncs, no bank feeds, no location tracking.

Input Type	Format	How It Enters
Text (typed)	Raw string	Capture UI text box, command palette, global shortcut
Voice memo	Audio → text	Whisper transcription (local or API) to text
Screenshot	Image → text	OCR + LLM visual understanding to structured text
Photo (whiteboard, document)	Image → text	SigLIP 2 (400M) lazy-loaded for image captures

Palantir Parallel

Palantir parallel: In Palantir Foundry, raw data enters through **Data Connections**—connectors to databases, APIs, file systems, and streaming sources. Each connector handles format-specific ingestion (CSV, JSON, Parquet, JDBC, etc.). Memora's input layer is the personal equivalent: instead of enterprise connectors, you have voice, text, and camera—three "connectors" to your life.

Design principle: Capture action must complete in <2 seconds. Zero-friction capture means the user never hesitates to record a thought. Every entry is timestamped, tagged with input modality, and assigned a unique content hash for deduplication.

3.2 Stage 2: Preprocessing

Before the Archivist agent sees the input, deterministic preprocessing normalizes the raw capture:

1. **Transcription** (voice only): Whisper model converts audio to text with speaker diarization
2. **OCR + Visual Understanding** (images only): Extract text via OCR; pass image + extracted text to LLM for structured interpretation ("This is a receipt from Starbucks for $5.40 on Feb 20")
3. **Text Normalization**: Standardize dates ("next Tuesday" → 2026-03-03), currency ("5 bucks" → $5.00), names ("Dr. Smith" → normalized form)
4. **Language Detection**: BGE-M3 supports 100+ languages; detect language for proper tokenization
5. **Deduplication Check**: Content hash compared against recent captures to prevent duplicate processing

Technical Detail

Preprocessing is **entirely deterministic**—no LLM involvement. This ensures reproducibility and keeps costs at zero for this stage. The LLM is only invoked at Stage 3 (Archivist Extraction).

3.3 Stage 3: The Archivist — LLM Extraction Engine

This is the critical stage where unstructured natural language becomes structured graph operations. The Archivist is the first AI agent in the pipeline and the highest-frequency agent in the system.

Agent Role

The Archivist is the graph construction engine. It runs on **every single capture**—the only agent that operates at input frequency. Recommended model: **Claude Haiku** (fast, cheap, structured output). With prompt caching (0.1× base cost for cache hits), Archivist costs drop 60–70% because the system prompt + graph schema context is near-identical across calls.

3.3.1 What the Archivist Does

Given a preprocessed text input, the Archivist performs:

1. **Entity Extraction**: Identify all entities mentioned—people, organizations, places, events, dates, financial amounts, commitments, decisions, goals, ideas, concepts
2. **Relationship Extraction**: Identify how entities relate to each other—"Sam *promised* to introduce me to *his investor*" yields: PERSON(Sam) –[PROMISED]–> COMMITMENT(introduction) –[INVOLVES]–> PERSON(investor)
3. **Attribute Extraction**: Pull properties for each entity—roles, amounts, dates, sentiments, confidence levels
4. **Network Classification**: Assign each entity and relationship to one or more of the seven context networks (Academic, Professional, Financial, Health, Personal Growth, Social, Ventures)
5. **Temporal Anchoring**: Attach temporal metadata—when did this happen? When is it due? Is this past, present, or future?
6. **Confidence Scoring**: Rate extraction confidence (0–1) for each proposed node and edge

3.3.2 Constrained Output via Pydantic Schemas

The Archivist does **not** return free-form text. It returns structured JSON validated against strict Pydantic schemas. This is the mechanism that prevents hallucinated graph structure.

Listing 1: Simplified Archivist output schema (Pydantic)

```python
class GraphProposal(BaseModel):
    """Atomic set of graph changes proposed by the Archivist."""
    source_capture_id: str
    timestamp: datetime
    confidence: float # 0.0 - 1.0 overall

    nodes_to_create: list[NodeProposal]
    nodes_to_update: list[NodeUpdate]
    edges_to_create: list[EdgeProposal]
    edges_to_update: list[EdgeUpdate]
```

```python
    network_assignments: list[NetworkAssignment]

class NodeProposal(BaseModel):
    """A proposed new node in the knowledge graph."""
    temp_id: str # temporary ID for edge references
    node_type: NodeType # enum: PERSON, EVENT, COMMITMENT, etc.
    title: str # human-readable name
    content: str # full extracted content
    properties: dict # type-specific properties
    confidence: float # extraction confidence
    networks: list[NetworkType] # which context networks
    temporal: TemporalAnchor | None # when does this apply

class EdgeProposal(BaseModel):
    """A proposed relationship between two nodes."""
    source_id: str # node temp_id or existing graph ID
    target_id: str
    edge_type: EdgeType # enum: 7 categories, 30+ subtypes
    edge_category: EdgeCategory # STRUCTURAL, ASSOCIATIVE, etc.
    properties: dict
    confidence: float
    bidirectional: bool
```

Palantir Parallel

Palantir parallel: This is directly analogous to Palantir's Pipeline Builder **Ontology Output** step. In Foundry, you add an "Ontology output" to your pipeline that maps transformed data columns to object type properties and link types. The Archivist's Pydantic schema is Memora's equivalent of Palantir's ontology mapping configuration—it defines exactly how unstructured information maps to typed graph elements.

3.3.3 The Archivist's System Prompt Architecture

The Archivist's system prompt is carefully engineered with:

1. **The complete graph schema**: All node types, edge types, property definitions, and valid combinations
2. **Existing entity context** (via RAG): Recent nodes that might be referenced or updated—preventing duplicate creation
3. **Network definitions**: What each context network represents, with examples of correct classification
4. **Extraction rules**: "If a person is mentioned by first name only and a matching PERSON node exists, reference the existing node rather than creating a new one"
5. **Clarification protocol**: If the input is ambiguous, the Archivist can request clarification from the user instead of guessing

Technical Detail

Prompt caching is critical here. The system prompt (schema + rules + network definitions) is ∼2,000–4,000 tokens and identical across 95%+ of calls. Only the RAG context window and the user's actual capture change. With Anthropic's prompt caching at 0.1× base cost, this means the expensive part of the prompt is cached, and only the variable suffix incurs full cost.

3.4 Stage 4: Entity Resolution

Entity resolution is the process of determining whether a newly extracted entity refers to an **existing** node in the graph or is genuinely new. This is the single hardest problem in the pipeline.

Palantir Parallel

Palantir parallel: Palantir's **Foundry Entity Resolution** is one of their flagship capabilities. It "seamlessly links records with AI to establish a reliable, de-duplicated data foundation." The output is a sequence of `(A, same_as, B)` triples, and in aggregate these form a graph where each connected component represents a single real-world entity. Palantir uses probabilistic matching, machine learning models, and human-in-the-loop review for ambiguous cases. Memora must solve the same problem at personal scale.

3.4.1 Resolution Strategy

Entity resolution in Memora uses a multi-signal approach:

1. **Exact match**: Content hash or normalized name matches an existing node → reference existing
2. **Semantic similarity**: Embedding cosine similarity > 0.92 against existing nodes of the same type → candidate match
3. **Contextual match**: The Archivist is given the top-K similar existing nodes as RAG context and asked: "Is this the same entity?" → LLM-assisted resolution
4. **Temporal + relational**: If "Sam" appears in the same context network, within the same time window, connected to the same people → high probability of match
5. **Ambiguity flag**: If confidence is between 0.6–0.85, flag for human review rather than auto-resolving

Signal	Weight	Details
Exact name match	0.95	Normalized canonical name comparison
Embedding similarity (>0.92)	0.80	BGE-M3 dense vector cosine similarity
Same context network	0.15	Bonus if in overlapping networks
Temporal proximity	0.10	Mentioned within 7-day window of existing node
Shared relationships	0.20	Connected to same PERSON/EVENT nodes
LLM adjudication	0.90	Archivist explicitly asked "same entity?"

3.4.2 Entity Resolution Outcomes

- **Merge**: Two nodes are confirmed as the same entity → merge properties, keep all edges, log the merge event
- **Create**: No match found → create new node with full provenance
- **Link**: Entities are related but distinct → create an edge (e.g., "Sam (investor)" and "Sam (friend)" might be the same person or might not)
- **Defer**: Ambiguous case → flag for human review with both candidates presented

3.5 Stage 5: Graph Proposal Assembly

After extraction and entity resolution, the Archivist assembles a complete **Graph Proposal**—an atomic set of changes to be applied to the knowledge graph. This is a structured diff, not a mutation.

Key Insight

Nothing enters the graph without a proposal. The graph proposal is the fundamental unit of change in Memora. It is analogous to a database transaction or a git commit—an atomic, reviewable, reversible set of changes. This is the governance mechanism that prevents graph corruption.

A graph proposal contains:

- **New nodes** to create (with all properties, confidence scores, network assignments)
- **Existing nodes** to update (property changes, confidence adjustments, new network memberships)
- **New edges** to create (typed relationships between new and/or existing nodes)
- **Existing edges** to update (relationship changes, weight adjustments)
- **Provenance chain**: which capture triggered this proposal, which agent created it, what extraction rules were applied
- **Overall confidence score**: weighted average of all individual confidences
- **Human-readable summary**: natural language description of what the proposal does ("Adding person Sam Chen, linking to existing Project Alpha, creating commitment: intro to investor by March 5")

Palantir Parallel

Palantir parallel: In Palantir's Foundry, Pipeline Builder outputs map to **Ontology outputs**—the final step where transformed data becomes ontology objects. The pipeline documentation recommends an "intermediate transform step from clean to ontology" to ensure data quality. Memora's Graph Proposal is this intermediate step—a validated, structured diff that has been through extraction, entity resolution, and schema validation before touching the graph.

3.6 Stage 6: Validation Gate

The validation gate routes proposals based on confidence and impact:

Technical Detail

Decision fatigue mitigation: Research shows quality degrades after ∼35 micro-decisions per day (Baumeister et al.). Auto-approve at ≥0.85 confidence means most routine captures flow through silently, while the daily review digest catches errors. Target: <5% of auto-approved changes require correction.

3.7 Stage 7: Human Review / Auto-Approve

Depending on the validation gate's routing:

- **Auto-approved proposals**: Committed immediately, logged in the daily review digest for retrospective correction
- **Digest-routed proposals**: Batched into the morning review—user sees a list of proposed changes and can approve, edit, or reject each one
- **Explicit-confirm proposals**: Presented immediately with full context—"The Archivist wants to merge Person(Sam) with Person(Samuel Chen). Here are both profiles. Approve?"

3.8 Stage 8: Graph Commit

Once approved (automatically or manually), the proposal is committed atomically:

1. **Transaction begin**: All node/edge creations and updates are wrapped in a single atomic transaction
2. **Node creation/update**: Nodes written to RyuGraph/DuckDB with all properties, timestamps, and approval status
3. **Edge creation/update**: Typed edges created between nodes with provenance metadata
4. **Provenance logging**: Full audit trail—which capture, which agent, which extraction rules, human approval status
5. **Transaction commit**: All-or-nothing—if any step fails, the entire proposal is rolled back

3.9 Stage 9: Post-Commit Processing

After a successful commit, several asynchronous processes fire:

1. **Embedding generation**: BGE-M3 generates dense + sparse embeddings for all new/updated nodes → written to LanceDB
2. **Bridge discovery** (incremental): New node's embedding is compared against nodes in *other* context networks via HNSW index lookup ($O(\log N)$, ∼1–5ms at 10K nodes). High-similarity cross-network pairs are flagged as potential bridges
3. **Network health recalculation**: If the new data affects commitment completion rates or alert counts, network health status is updated
4. **Notification triggers**: Check if the new data triggers any notification rules (deadline approaching, relationship decay, goal drift)
5. **Truth Layer cross-reference**: If the committed data contains claims that intersect with verified facts, flag any contradictions

4 Graph Construction: The Ontology

4.1 Node Types (The Entities)

Memora's graph uses **12 typed node categories** organized into two clusters, directly analogous to Palantir's object types:

Complete node type taxonomy. Life Context Nodes represent things that *happen* in your life. Knowledge Nodes represent things you *know* or *think*. Cross-connections (gold arrows) between these clusters are where the highest-value insights live.

4.2 Shared Node Properties (The Metadata Layer)

Every node in the graph carries a standard set of properties—the metadata that enables search, decay, confidence tracking, and provenance:

Property	Type	Purpose
`id`	UUID	Unique identifier, immutable after creation
`content_hash`	SHA-256	Deduplication key—identical content = identical hash
`created_at`	Timestamp	When this node was first created
`updated_at`	Timestamp	Last modification timestamp
`embedding`	Float[1024]	BGE-M3 dense vector for semantic search
`sparse_embedding`	Sparse vector	BGE-M3 BM25-compatible sparse representation
`confidence`	Float [0–1]	Extraction confidence from the Archivist
`networks`	List[NetworkType]	Which context networks this node belongs to
`human_approved`	Boolean	Whether a human has reviewed and approved this node
`proposed_by`	AgentID	Which agent proposed this node (Archivist, Researcher, etc.)
`source_capture_id`	UUID	Link back to the original raw capture
`access_count`	Integer	How many times this node has been accessed (for decay)
`last_accessed`	Timestamp	Last time this node was referenced (for decay)
`decay_score`	Float	Current decay score (exponential decay function)
`review_date`	Timestamp	When this node is due for spaced repetition review
`tags`	List[String]	User-defined and auto-generated tags

Palantir Parallel

Palantir parallel: Every Palantir object type has **properties** that define its characteristics, with support for shared properties across types, value formatting, conditional formatting, and mandatory control properties. Memora's shared node properties serve the same function—they are the "shared property set" that every node type inherits, ensuring consistent metadata for search, governance, and analytics across the entire graph.

4.3 Edge Types (The Relationships)

Edges are typed across **seven categories**, each serving a different analytical purpose:

Category	Example Subtypes	What It Captures
STRUCTURAL	PART_OF, CONTAINS, SUBTASK_OF	Hierarchy and composition
ASSOCIATIVE	RELATED_TO, INSPIRED_BY, CONTRADICTS	Semantic relationships
PROVENANCE	DERIVED_FROM, VERIFIED_BY, SOURCE_OF	Where information came from
TEMPORAL	PRECEDED_BY, EVOLVED_INTO, TRIGGERED	Time-ordered causation
PERSONAL	COMMITTED_TO, DECIDED, FELT_ABOUT	Personal stakes and emotions
SOCIAL	KNOWS, INTRODUCED_BY, OWES_FAVOR	People and social dynamics
NETWORK	BRIDGES, MEMBER_OF, IMPACTS	Cross-network connections

Palantir Parallel

Palantir parallel: Palantir's **Link Types** are "the schema definition of a relationship between two object types." Law enforcement users, for example, define link types like "communicated_with," "traveled_to," "associated_with," and "witnessed_by." Memora's seven edge categories serve the same function but for personal life: SOCIAL edges track who you know and owe favors to, TEMPORAL edges track how ideas evolved, and NETWORK edges (the "bridges") are the cross-domain connections that represent the highest-value intelligence—exactly like Palantir's link analysis finding hidden connections between suspects across databases.

4.4 Context Networks (The Subgraphs)

The graph is organized into **seven interconnected context networks**—living subgraphs that each track a domain of the user's life:

Network	What It Tracks
Academic	Courses, grades, research, academic relationships, study commitments
Professional	Work projects, clients, colleagues, career goals, deliverables
Financial	Transactions, budgets, investments, financial commitments
Health	Exercise, sleep, stress, medical appointments, mental health patterns
Personal Growth	Learning goals, skills, habits, self-improvement
Social	Friends, family, social events, relationship health
Ventures	Side projects, entrepreneurial activities, business ideas

Critical design point: A single node can belong to **multiple networks**. "Sam" might be in both Professional and Social. A COMMITMENT to "submit the proposal" might be in both

Academic and Ventures. These multi-membership nodes are the natural bridge points where cross-domain intelligence emerges.

5 How AI Agents Operate on the Graph

5.1 The Agent Architecture: Palantir AIP vs. Memora Council

Palantir Parallel

Palantir's **AIP Agent Studio** builds agents with three components: (1) **Context sources** — ontology context (structured objects via semantic search), document context (unstructured text), and function-backed context (custom retrieval); (2) **Tools** — actions (write to ontology), object queries (read from ontology), custom functions; (3) **LLM backbone** — commercial models (GPT, Claude, Gemini) or open-source (Llama), connected to the ontology via the platform's security model.
Memora's AI Council mirrors this architecture exactly, with three specialized agents instead of a general-purpose agent framework.

5.2 Agent 1: The Archivist (Graph Writer)

The Archivist agent: receives raw capture + RAG context of existing nodes + graph schema constraints, outputs a structured graph proposal. This mirrors Palantir AIP's agent pattern of retrieval context + tools + LLM generation.

Frequency: Every capture (10+/day). **Model:** Claude Haiku (fast, cheap, structured). **Context sources:** Graph schema (static), recent nodes via RAG (dynamic), user's capture (input). **Tools:** Graph query (read existing nodes for entity resolution), schema validator (check proposal validity). **Output:** Structured GraphProposal (Pydantic-validated JSON).

5.3 Agent 2: The Strategist (Graph Reader + Analyst)

The Strategist is the **intelligence analyst** of Memora—the agent that reads the graph, computes cross-network analysis, and delivers actionable intelligence.

Agent Role

The Strategist operates at two frequencies: (1) **Daily**—compiles the morning briefing from graph state; (2) **On-demand**—when the user asks a complex question that requires cross-domain reasoning. It also includes a **Critic mode** for adversarial thinking ("What am I missing? What could go wrong?").

What the Strategist does:

1. **Cross-network bridge analysis**: Uses bridge discovery results to identify patterns across life domains ("Your stress mentions correlate with your open commitment count—you're overextended")
2. **Network health assessment**: Reads computed health scores, momentum, and alerts for each context network
3. **Decision recommendations**: Given a user's question + graph context + Truth Layer facts, recommends specific actions with rationale
4. **Priority ranking**: Orders today's tasks by urgency, importance, and cross-network impact

5. **Temporal pattern detection**: Identifies trends over time ("Your Academic network has been declining for 3 weeks")
6. **Critic mode**: Deliberately challenges the user's assumptions and decisions ("You're assuming the investor will follow through, but your graph shows Sam has missed 2 prior commitments")

Palantir Parallel

Palantir parallel: The Strategist is Memora's version of a Palantir Gotham **intelligence analyst workflow**. In Gotham, analysts use the platform to "map social networks, track movements, identify physical characteristics, and review history" of entities. The Strategist does the same for your personal entities—tracking commitment fulfillment rates, relationship health, goal progress, and cross-domain correlations. The Critic mode parallels Palantir's use in **red-teaming and adversarial analysis**—deliberately looking for what the primary analysis missed.

5.4 Agent 3: The Researcher (Internet Bridge)

Agent Role

The Researcher is the agent that bridges your private graph with the public internet. It operates **on-demand** when the Strategist or user needs external information to make a decision. It deposits verified facts into the Truth Layer.

Tools available (via MCP servers):

- **Google Search** (100 free queries/day)
- **Brave Search** (2,000 free/month, fallback)
- **Playwright** (full web scraping for deep research)
- **Semantic Scholar + arXiv** (academic paper search)
- **GitHub MCP** (code and repository search)
- **Graph + Vector DB** (query Memora's own graph as an agent tool)

Critical constraint: The Researcher must **anonymize graph context** before sending external queries. It cannot leak personal information to search engines. Query construction strips PII and uses abstract patterns.

5.5 The Orchestrator: Agent Coordination

The AI Council architecture with orchestrator coordination. Each agent has a specific role, specific data access, and specific frequency. The orchestrator decomposes complex queries, routes to the appropriate agent(s), and synthesizes results.

5.6 Council Decision Pattern (High-Stakes Queries)

When you ask Memora a complex question like "Should I take this job offer?", the council follows a structured deliberation:

1. **Decomposition**: Orchestrator breaks the question into sub-queries:

- Archivist: "What is the user's current professional context, commitments, and financial situation?"
- Strategist: "What cross-network impacts would this decision have? What's the user's risk profile?"
- Researcher: "What are current market conditions for this role? Company reputation? Glassdoor data?"

2. **Independent Analysis**: Each agent works in parallel over different data sources
3. **Proposal Submission**: Each submits: answer, confidence (0–1), evidence (graph nodes + sources), and risks
4. **Deliberation** (optional, max 2–3 rounds): Agents review each other's proposals for contradictions
5. **Synthesis**: Confidence-weighted aggregation. High disagreement flagged for human review

6 The Truth Layer: Fact Verification Pipeline

6.1 Why It Exists

The graph contains three categories of information with different reliability:

Source Type	Reliability	Examples
PRIMARY	Highest	Official records, published financials, government data
SECONDARY	Medium	News articles, research papers, third-party reports
SELF_REPORTED	Lowest	User hearsay ("Sam told me X"), personal observations

The risk: If the Strategist recommends an action based on a hallucinated LLM inference layered on top of stale self-reported hearsay, you've made a decision on fiction. The Truth Layer prevents this.

6.2 The Fact-Check Gate

Before any agent generates a recommendation, it cross-references against the Truth Layer:

- **Contradiction found**: Flag to user, show both claims with sources, agent must acknowledge
- **No verified facts exist**: Proceed with caveat—"unverified claim"
- **Facts support claim**: Proceed with citation to verified source

Palantir Parallel

Palantir parallel: Palantir's platform includes extensive **data lineage and provenance tracking**. Every piece of data in Foundry has a complete audit trail—where it came from, how it was transformed, when it was last updated. The Truth Layer is Memora's equivalent: a verified fact store with source typing, staleness detection, and contradiction resolution. In intelligence work, the principle is "trust but verify"—the Truth Layer automates the "verify" step.

7 Background Mechanics: The Living Graph Engine

These are the **deterministic, LLM-independent algorithms** that run on schedule—the "engine room" that transforms a static graph into a living surveillance system.

7.1 Decay Scoring (Daily)

Every node decays exponentially based on time since last access or mention. The decay function:

$$\text{decay_score}(t) = e^{-\lambda \cdot (t_{\text{now}} - t_{\text{last_access}})}$$

where λ is the decay constant (configurable per network). Nodes with low decay scores are candidates for resurfacing via spaced repetition or archival.

7.2 Bridge Discovery (Daily + Per-Capture)

Bridge discovery is the mechanism that finds **cross-network connections**—the highest-value intelligence in the system.

1. **Per-capture (incremental)**: When a new node is committed, its embedding is compared via HNSW index against nodes in *other* networks. Cost: $O(\log N)$, ~1–5ms at 10K nodes.
2. **Daily batch**: Nodes modified in the last 24 hours are batch-scanned for cross-network bridges. High-similarity pairs are batched into a single LLM call ("Here are 15 potential connections—which are meaningful?")—1 API call instead of 15.

Palantir Parallel

Palantir parallel: This is directly analogous to Palantir Gotham's **link analysis** capability. In intelligence work, link analysis discovers hidden connections between entities across databases—a phone number appearing in two unrelated cases, a financial transaction linking two suspects. Bridge discovery is Memora's personal link analysis: finding that your stress (Health network) correlates with your open commitment count (Professional network), or that a contact in your Social network is the perfect investor for your Ventures network.

7.3 Network Health Scoring (Every 6 Hours)

Each context network has a computed health status:

- **On Track**: Commitments met, no overdue items, recent engagement
- **Needs Attention**: Some open alerts, declining momentum, stale commitments
- **Falling Behind**: Multiple overdue items, no recent input, high alert count

Computed from: commitment completion rate (actionable), alert ratio (urgency), staleness flags (only triggered when commitments exist but haven't been updated).

7.4 Spaced Repetition (Daily)

The SM-2 algorithm surfaces knowledge nodes due for review. This is how Memora fights the "67% of notes never revisited" problem—important knowledge is actively resurfaced on a scientifically-optimized schedule.

7.5 Gap Detection (Weekly)

Identifies structural weaknesses in the graph:

- **Orphaned nodes**: Nodes with no edges (isolated information)
- **Stalled goals**: GOAL nodes with no recent PROGRESS edges
- **Dead-end projects**: PROJECT nodes with no recent activity
- **Isolated concepts**: CONCEPT nodes not linked to any practical application

8 Palantir-Class Surveillance for Personal Intelligence

8.1 The Surveillance Analogy

Palantir's platform has been described as a "secondary surveillance network" that "extensively catalogs and maps interpersonal relationships between individuals." By cross-referencing databases, it creates "complex digital profiles for any individual"—tracking communication data, movement profiles, and social connections in a single system.

Memora applies the same surveillance philosophy **to yourself**. You are simultaneously the intelligence agency and the subject. The "databases" being cross-referenced are your own life domains.

8.2 What Memora Surveils

Domain	Palantir (Government)	Memora (Personal)
Entity Tracking	Suspects, persons of interest	People in your life, their commitments to you
Relationship Mapping	Social networks, communication patterns	Who knows whom, favor debts, introduction chains
Commitment Monitoring	Compliance, legal obligations	Promises made (by you and to you), deadlines
Temporal Analysis	Movement patterns, event timelines	How your ideas evolve, when decisions were made
Anomaly Detection	Unusual transactions, behavioral changes	Stress spikes, network health drops, goal drift
Cross-Domain Intelligence	Linking financial + communication + travel	Linking health + professional + financial + social
Proactive Alerts	Threat warnings, risk assessments	Deadline alerts, relationship decay, opportunity flags
Daily Briefing	Intelligence briefing for analysts	Morning life situation report for you

8.3 The Daily Briefing as Intelligence Report

Every morning, the Strategist compiles a **life situation report**—Memora's equivalent of a Palantir intelligence briefing:

1. **Network Status**: Per-network health (On Track / Needs Attention / Falling Behind) with momentum direction
2. **Open Alerts**: Ranked by urgency—approaching deadlines, decaying relationships, stale commitments
3. **Cross-Network Bridges**: New correlations discovered in the last 24 hours
4. **Decision Prompts**: Questions the system thinks you should answer today
5. **Recommended Actions**: People to contact, deadlines to renegotiate, opportunities to pursue
6. **Spaced Repetition**: Knowledge nodes due for review

8.4 The Web of Connections

The fundamental power of both Palantir and Memora is the ability to see **connections that are invisible when information is siloed**:

Key Insight

Example intelligence chain:
You mention stress twice this week (Health network) → You have 4 open commitments past due (Professional network) → One of those commitments involves a person who also appears in your Ventures network → That person recently missed a commitment to you (Social network) → The Strategist surfaces: "You're stressed because you're overextended. The commitment from Sam is stale—he missed his last two promises. Consider deprioritizing the venture with Sam and focusing on your 3 remaining professional commitments."
No single-domain tool can see this chain. Only a cross-domain knowledge graph with AI reasoning can connect stress → overcommitment → unreliable contact → actionable recommendation.

9 The Adaptive RAG Pipeline

When you query Memora ("What's my situation with the startup funding?"), the retrieval system doesn't just do vector search. It routes adaptively based on query complexity:

Query Type	Retrieval Strategy	Example
Simple factual	Vector search only (LanceDB)	"When did I meet Sam?"
Relationship	Graph traversal + vector merge	"Who introduced me to the VC?"
Cross-network	Multi-network graph walk	"How is my health affecting my work?"
Complex decision	Multi-step agentic RAG	"Should I take this job offer?"

All paths converge through: **Hybrid Search** (BM25 + dense vector fusion) → **CRAG Quality Check** (if retrieval quality is poor, fall back to Researcher agent for web search) → **Graph-Augmented Context Expansion** (expand retrieved nodes to include neighboring context) → **LLM Generation with Source Citations**.

Palantir Parallel

Palantir parallel: Palantir AIP agents use **retrieval context** with three modes: ontology context (semantic search over objects), document context (semantic search over text chunks), and function-backed context (custom retrieval). The agent automatically determines which context is most relevant. Memora's adaptive RAG router serves the same purpose—classifying query complexity and routing to the appropriate retrieval strategy, from simple vector search to full agentic multi-step reasoning.

10 Technical Architecture Comparison: Palantir vs. Memora

Architecture Layer	Palantir	Memora
Data Ingestion	Data Connections (100s of enterprise connectors, JDBC, APIs, file systems, streaming)	Three input modalities (text, voice/Whisper, image/OCR+SigLIP)
Data Pipeline	Foundry Pipeline Builder (Spark + Flink transforms, clean → ontology intermediate step)	Archivist extraction pipeline (preprocessing → LLM extraction → entity resolution → proposal)
Ontology / Schema	Object Types + Properties + Link Types (Semantic Layer), Actions + Functions (Kinetic Layer), Security (Dynamic Layer)	12 Node Types + 7 Edge Categories + Shared Properties, Pydantic schema validation
Entity Resolution	Foundry Entity Resolution (AI-powered record linkage, produces same_as triples, connected components = single entity)	Multi-signal resolution (exact match, embedding similarity, contextual LLM, temporal + relational, ambiguity flagging)
Graph Database	Proprietary (Palantir's internal graph infrastructure)	RyuGraph / DuckDB (embedded, zero-cost, Cypher-compatible)
Vector Search	Integrated into Ontology (vector embedding properties on object types)	LanceDB (embedded, HNSW index, BGE-M3 dense + sparse)
AI Agents	AIP Agent Studio (LLM + ontology context + document context + tools, native/prompted tool calling)	AI Council (3 agents: Archivist/Strategist/Researcher, LangGraph orchestrator)
Agent Tools	Ontology actions (write), object queries (read), custom functions	MCP servers (Google Search, Brave, Playwright, arXiv, GitHub, graph/vector DB)
Governance	Dynamic security layer, markings, classifications, audit trails	Validation gate (auto-approve / digest / explicit confirm), provenance logging
Intelligence Output	Dashboards, alerts, operational decisions, threat detection	Daily briefings, bridge discovery, decision recommendations, critic mode
Deployment	Cloud (Palantir-hosted or customer cloud, SaaS or on-prem)	Local-first (embedded DBs, single machine, $0 infrastructure)
Cost	Enterprise contracts ($M+/year)	$0 infra + $10–15/month BYOK LLM API

11 Conclusion: Building Your Personal Intelligence Agency

Key Insight

Memora is architecturally a **personal Palantir**. The pipeline is identical:

1. **Ingest** raw data from multiple sources (text, voice, images)
2. **Transform** unstructured input into structured entities via AI extraction (Archivist = Foundry Pipeline)
3. **Resolve** entities across captures to prevent duplication (Entity Resolution)
4. **Map** to a typed ontology with nodes, edges, and properties (Graph Schema = Palantir Ontology)
5. **Validate** through governance gates before committing (Validation Gate = Dynamic Security Layer)
6. **Commit** atomically to the knowledge graph (Graph Commit = Ontology Write)
7. **Enrich** with embeddings, bridge discovery, and post-commit processing (Post-Commit = Foundry Enrichment)
8. **Deploy agents** that read/write the graph with verified context (AI Council = AIP Agents)
9. **Surface intelligence** through daily briefings, alerts, and decision recommendations (Briefing = Intelligence Report)
10. **Verify facts** against a curated truth store before acting (Truth Layer = Data Lineage + Provenance)

The difference: Palantir surveils organizations and nations at enterprise scale for $M+/year. Memora surveils your own life at personal scale for $15/month. The surveillance target is yourself. The intelligence consumer is yourself. The decisions being optimized are yours.

Every connection Palantir finds between suspects across databases, Memora finds between your commitments across life domains. Every threat Palantir flags for an intelligence analyst, Memora flags for you—approaching deadlines, decaying relationships, drifting goals, hidden opportunities.

This is not a note-taking app. This is a **personal intelligence platform**.

End of Document — Memora Input-to-Graph Deep Dive v1.0